
PML

UNIT III: Notes

(Note: Python Programming Language has been used for codes and Google Colab/Jupyter/python installed on system as interfaces, Only half unit(highlighted) is considered for CAE-I)

Prediction with Tensorflow

Train, tune and use time series, sequence and prediction models. Train models to predict values for both univariate and multivariate time series, Prepare data for time series learning, Understand Mean Absolute Error (MAE) and how it can be used to evaluate accuracy of sequence models, Use RNNs and CNNs for time series, sequence and forecasting models, Identify when to use trailing versus centered windows, Identify and compensate for sequence bias.

Training, Tuning, and Using Time Series, Sequence, and Prediction Models

1. Introduction

Objective: Train, tune, and use models for predicting future values in time series and sequences. This involves various techniques and approaches tailored to handle temporal dependencies and sequential data.

1. Training Time Series, Sequence, and Prediction Models

Time Series Models

1. Preparation:

- **Data Collection:** Gather historical time series data.
- **Data Cleaning:** Handle missing values, outliers, and errors.
- **Normalization:** Scale data to a range (e.g., 0 to 1).

2. Model Building:

- **ARIMA Model:**
 - **AutoRegressive (AR):** Uses lagged values.
 - **Integration (I):** Makes data stationary by differencing.

- **Moving Average (MA):** Uses past forecast errors.
- **Exponential Smoothing:**
 - **Simple Exponential Smoothing:** Weighted average of past observations.
 - **Holt's Linear Trend Model:** Extends simple smoothing to capture trends.
 - **Holt-Winters Seasonal Model:** Captures seasonality in addition to trend.

3. Training:

- **Fit Model:** Apply training data to the chosen model.
- **Validation:** Use part of the data to validate the model's performance.

Sequence Models

1. Preparation:

- **Data Collection:** Gather sequential data (e.g., text, user actions).
- **Sequence Padding:** Ensure sequences are of the same length for input.

2. Model Building:

- **Recurrent Neural Networks (RNNs):** Suitable for capturing temporal dependencies.
- **Long Short-Term Memory (LSTM) Networks:** Handle long-term dependencies and complex interactions.

3. Training:

- **Compile Model:** Choose appropriate loss function and optimizer.
- **Fit Model:** Train with sequences and corresponding targets.

Prediction Models

1. Preparation:

- **Feature Engineering:** Extract relevant features for prediction.
- **Data Split:** Divide into training, validation, and test sets.

2. Model Building:

- **Linear Regression:** For continuous values.
- **Logistic Regression:** For binary classification.
- **Decision Trees/Random Forests:** Handle both regression and classification.
- **Neural Networks:** For complex patterns in large datasets.

3. Training:

- **Compile and Fit:** Use training data to train the model.
 - **Hyperparameter Tuning:** Adjust parameters (e.g., learning rate, number of layers).
-

2. Tuning Time Series, Sequence, and Prediction Models

Time Series Models

1. Hyperparameter Tuning:

- **ARIMA:** Select optimal values for p (AR order), d (I order), and q (MA order).
- **Exponential Smoothing:** Adjust smoothing parameters (alpha, beta, gamma).

2. Model Selection:

- **Evaluate Performance:** Use metrics like Mean Absolute Error (MAE) or Root Mean Squared Error (RMSE).

Sequence Models

1. Hyperparameter Tuning:

- **LSTM Units:** Number of LSTM units or layers.
- **Dropout Rate:** Prevent overfitting.
- **Learning Rate:** Adjust optimizer's learning rate.

2. Model Selection:

- **Cross-Validation:** Use techniques like k-fold cross-validation for robust evaluation.
- **Evaluate Performance:** Metrics like accuracy, F1-score, or loss.

Prediction Models

1. Hyperparameter Tuning:

- **Learning Rate:** For gradient-based models.
- **Number of Epochs:** Training iterations.
- **Batch Size:** Size of data batches for training.

2. Model Selection:

- **Evaluate Metrics:** Use appropriate metrics based on the problem (e.g., MAE for regression, accuracy for classification).

3. Using Time Series, Sequence, and Prediction Models

Time Series Models

1. **Make Predictions:**
 - **Forecasting Future Values:** Apply the trained model to predict future data points.
 - **Confidence Intervals:** Provide uncertainty estimates for forecasts.
2. **Deployment:**
 - **Integrate with Applications:** Use forecasts in decision-making processes.

Sequence Models

1. **Generate Sequences:**
 - **Predict Next Elements:** For tasks like text generation or sequence continuation.
2. **Deployment:**
 - **Real-Time Predictions:** Integrate models in applications for real-time sequence analysis.

Prediction Models

1. **Make Predictions:**
 - **Generate Outputs:** Use trained models to generate predictions for new data.
2. **Deployment:**
 - **Operationalize:** Integrate into systems for real-world usage (e.g., recommendation systems, automated responses).

Univariate vs. Multivariate Time Series Comparison

1. Definition

Univariate Time Series:

- **Definition:** A time series that consists of observations of a single variable over time.
- **Example:** Daily temperature readings, monthly sales figures.

Multivariate Time Series:

- **Definition:** A time series that consists of observations of multiple variables over time.
- **Example:** Daily temperature, humidity, and wind speed readings; monthly sales figures across multiple products.

2. Characteristics

Feature	Univariate Time Series	Multivariate Time Series
Data Points	Single variable per time point.	Multiple variables per time point.
Complexity	Generally simpler to analyze and model.	More complex due to the interactions between variables.
Modeling	Models focus on the temporal dynamics of a single variable.	Models must capture both temporal dynamics and inter-variable relationships.
Example	Predicting future sales based on past sales data.	Predicting future sales based on past sales, advertising spend, and economic indicators.

4. Comparison of Methods and Techniques

Aspect	Univariate Time Series	Multivariate Time Series
Data Preparation	Simpler, fewer variables to preprocess and visualize.	More complex, requires preprocessing and handling of multiple variables.
Feature Engineering	Focuses on creating lagged variables and trends for one variable.	Involves creating interactions, lagged variables, and trends for multiple variables.
Model Complexity	Generally simpler models like ARIMA, Exponential Smoothing.	More complex models like Vector Autoregression (VAR), Multivariate Regression, LSTM networks.
Visualization	Single plot to visualize data over time.	Multiple plots or multivariate plots to visualize relationships between variables over time.
Example Models	ARIMA, Exponential Smoothing, Linear Regression.	Vector Autoregression (VAR), Multivariate Regression, LSTM, GRU.

5. Model Selection

Univariate Models:

- **ARIMA (AutoRegressive Integrated Moving Average):** Models the dependency between an observation and some number of lagged observations.
- **Exponential Smoothing:** Forecasts based on weighted averages of past observations.

Components of ARIMA Explained with Simple Example

1. AutoRegression (AR)

- **Definition:** AutoRegression uses the relationship between an observation and several previous observations (lags) to make predictions.
- **Example:** If you have temperature data, an AR model might predict today's temperature based on the temperatures of the previous few days.

Simple Example: Suppose you have daily temperatures for the past week:

- Day 1: 30°C
- Day 2: 32°C
- Day 3: 31°C
- Day 4: 33°C
- Day 5: 35°C
- Day 6: 34°C
- Day 7: 36°C

An AR model might use the temperatures from Day 6 and Day 7 to predict the temperature for Day 8.

2. Integration (I)

- **Definition:** Integration involves differencing the data to make it stationary, which means its statistical properties (like mean and variance) do not change over time.
- **Example:** If the temperature data shows a trend (e.g., increasing temperatures), differencing the data will remove this trend, making it easier to model.

Simple Example: Suppose you notice the temperature is increasing by 1°C each day:

- Day 1: 30°C
- Day 2: 31°C
- Day 3: 32°C
- Day 4: 33°C
- Day 5: 34°C

- Day 6: 35°C
- Day 7: 36°C

The differences (Integration) are:

- Day 2 - Day 1 = 1°C
- Day 3 - Day 2 = 1°C
- Day 4 - Day 3 = 1°C
- Day 5 - Day 4 = 1°C
- Day 6 - Day 5 = 1°C
- Day 7 - Day 6 = 1°C

All differences are 1°C, showing a trend. Integration removes this trend, making the data stationary.

3. Moving Average (MA)

- **Definition:** Moving Average uses past forecast errors in a regression-like model. It captures the relationship between an observation and a residual error from a moving average model applied to lagged observations.
- **Example:** In temperature forecasting, an MA model might predict today's temperature based on the error (difference) between the observed and predicted temperatures from previous days.

Simple Example: Suppose you predicted the temperature for Day 7 to be 35°C, but the actual temperature was 36°C. The error (residual) is 1°C.

An MA model might use this error to adjust the prediction for Day 8.

Multivariate Models:

- **Vector Autoregression (VAR):** Extends ARIMA to handle multivariate time series.
- **Multivariate Regression:** Captures relationships between multiple independent and dependent variables.
- **LSTM (Long Short-Term Memory) Networks:** Handles long-term dependencies in sequential data, useful for complex interactions in multivariate data.

Preparing Data for Time Series Learning in TensorFlow

To build effective time series models in TensorFlow, it's essential to follow a systematic approach to data preparation. Here's a step-by-step guide:

1. Data Collection and Cleaning

Collect Data:

- Gather time series data from relevant sources such as sensors, historical records, or APIs.
 - **Example:** Daily temperature readings, stock prices, or sales data.

Clean Data:

- Address missing values, outliers, and errors to ensure data accuracy.
 - **Example:** Fill missing values using methods like interpolation or forward filling. Remove or correct outliers.

Purpose: Ensure the data is accurate, consistent, and ready for analysis.

2. Normalization

Normalize Data:

- Scale values to a consistent range (e.g., 0 to 1) for improved model performance.
 - **Example:** Normalize the temperature data to the range [0, 1] using min-max scaling.

Purpose: Scale the data to a consistent range, improving the performance and stability of the model.

python

Copy code

```
import numpy as np
```



```
def normalize(series):  
    min_val = np.min(series)  
    max_val = np.max(series)  
    return (series - min_val) / (max_val - min_val)
```

Example usage

```
data = np.array([30, 32, 31, 29, 35, 36, 34])  
normalized_data = normalize(data)  
print(normalized_data)
```

3. Sequence Creation

Create Sequences:

- Convert data into fixed-length sequences to predict future values. Define a window size for this purpose.
 - **Example:** Use a window size of 3 to create sequences of 3 days of temperature data to predict the 4th day's temperature.

Purpose: Convert the raw data into a format suitable for training a time series model.

python

Copy code

```
def create_sequences(data, window_size):  
    sequences = []  
    labels = []  
    for i in range(len(data) - window_size):  
        sequences.append(data[i:i + window_size])
```

```
        labels.append(data[i + window_size])

    return np.array(sequences), np.array(labels)

# Example usage

window_size = 3

X, y = create_sequences(normalized_data, window_size)

print("Sequences:", X)

print("Labels:", y)
```

4. Splitting Data

Train-Test Split:

- Divide data into training and testing sets, maintaining temporal order to avoid data leakage.
 - **Example:** Use the first 80% of the sequences for training and the remaining 20% for testing.

Purpose: Evaluate the model's performance on unseen data and prevent overfitting.

python

Copy code

```
split_ratio = 0.8

split_index = int(len(X) * split_ratio)

X_train, X_test = X[:split_index], X[split_index:]

y_train, y_test = y[:split_index], y[split_index:]
```

5. Batching and Shuffling

Batching:

- Group sequences into batches for efficient processing during training.
 - **Example:** Use a batch size of 32.

Shuffling:

- Randomize training data to reduce bias, but preserve temporal order within sequences.
 - **Note:** Do not shuffle the entire dataset as it will disrupt the temporal dependencies.

Purpose: Enhance the efficiency and generalizability of the model training process.

python

Copy code

```
import tensorflow as tf

batch_size = 32

train_dataset = tf.data.Dataset.from_tensor_slices((X_train, y_train))

train_dataset =
train_dataset.shuffle(buffer_size=len(X_train)).batch(batch_size)

test_dataset = tf.data.Dataset.from_tensor_slices((X_test,
y_test)).batch(batch_size)
```

6. Dataset Creation

Convert to TensorFlow Dataset:

- Use `tf.data.Dataset` to manage and preprocess data for training.
 - **Example:** Use the `from_tensor_slices` method to convert numpy arrays to TensorFlow datasets.

Purpose: Utilize TensorFlow's `tf.data.Dataset` API for efficient data handling and preprocessing.

python

Copy code

```
train_dataset = tf.data.Dataset.from_tensor_slices((X_train, y_train))

train_dataset =
train_dataset.shuffle(buffer_size=len(X_train)).batch(batch_size)

test_dataset = tf.data.Dataset.from_tensor_slices((X_test,
y_test)).batch(batch_size)
```

7. Feature Engineering (Optional)

- **Add Additional Features:** Create new features that might help the model.
 - **Example:** Adding moving averages, day of the week, or seasonality indicators to the dataset.

Understanding Mean Absolute Error (MAE) and its Use in Evaluating Sequence Model Accuracy

What is Mean Absolute Error (MAE)?

Mean Absolute Error (MAE) is a metric used to evaluate the accuracy of a regression model. It measures the average of the absolute differences between actual and predicted values. In simpler terms, it tells us how close the predictions are to the real values, on average.

Key Points:

- **MAE** is the average of the absolute errors (i.e., the differences between predicted and actual values).
- It is commonly used for evaluating regression models, including sequence models (like time series models).
- **Lower MAE** values indicate that the model's predictions are closer to the actual values, meaning the model is more accurate.
- It is **easy to interpret** and provides a direct measure of error.

Formula for MAE:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\text{Actual}_i - \text{Predicted}_i|$$

- n is the number of data points.
- Actual_i represents the actual value for the i -th data point.
- Predicted_i is the predicted value for the i -th data point.

Where

Use of MAE in Sequence Models:

In sequence models (such as time series forecasting), MAE is used to evaluate how well the model's predictions match the actual sequence. Since sequence models predict future points based on historical data, MAE helps in understanding the average deviation between the predicted sequence and the actual sequence.

Example: Calculating MAE

Let's take a simple example to illustrate how to calculate MAE and use it to evaluate the performance of a sequence model.

Given Data:

- **Actual Values:** [250, 245, 260, 255, 248]
- **Predicted Values:** [252, 244, 262, 254, 250]

Step-by-Step Calculation of MAE:

1. Find the absolute differences between each pair of actual and predicted values.

$$|250 - 252| = 2$$

$$|245 - 244| = 1$$

$$|260 - 262| = 2$$

$$|255 - 254| = 1$$

$$|248 - 250| = 2$$

2. Sum these absolute differences:

$$2 + 1 + 2 + 1 + 2 = 8$$

3. Divide by the number of data points ($n = 5$) to find the average:

$$\text{MAE} = \frac{8}{5} = 1.6$$

1.

Conclusion:

The MAE for the given data is **1.6**, meaning that, on average, the model's predictions deviate from the actual values by **1.6 units**. This value gives us a straightforward understanding of how well the model is performing: the closer the MAE is to 0, the better the model's accuracy.

Why Use MAE?

- **Intuitive:** MAE is easy to interpret because it tells you the average error in the same units as the data.
- **Robust to Outliers:** MAE is less sensitive to outliers compared to other metrics like Mean Squared Error (MSE), making it a good choice when you expect noise or anomalies in the data.
- **Direct Error Measurement:** It directly measures the average magnitude of errors in predictions, without considering their direction (i.e., overestimation or underestimation).

Conclusion:

Mean Absolute Error (MAE) is a useful and simple metric to evaluate the accuracy of regression models, especially for sequence models where predictions are made based on historical data. It provides an average measure of how far off predictions are from the actual values, making it a valuable tool for model evaluation and tuning.

CAE 2 Sullabus from here

RNNs and CNNs for Time Series and Sequence Models

1. Introduction to Time Series and Sequence Models

- **What are Time Series and Sequence Data?**
 - Time series data is a sequence of data points recorded at specific intervals (e.g., stock prices, temperature readings).
 - Sequence data refers to data that is arranged in a specific order (e.g., text data, audio).
- **Why Do We Need Specialized Models?**
 - Standard models don't work well because they can't account for the temporal relationships between data points in a sequence.

Example: Predicting the next day's temperature based on the past week's temperatures (time series).

2. Recurrent Neural Networks (RNNs)

- **What are RNNs?**
 - RNNs are a type of neural network designed to handle sequential data by maintaining a 'memory' of previous inputs through hidden states.
 - They are called "recurrent" because they perform the same task for every element of a sequence, with the output being dependent on previous computations.
- **How RNNs Work**

- At each step, an RNN takes in the current input and the hidden state (which stores information from previous inputs) and produces an output and a new hidden state.
- This allows the RNN to remember information from earlier in the sequence and use it to influence later predictions.
- **Challenges with RNNs**
 - **Vanishing Gradient Problem:** As the network goes through many layers, the gradients (used to update weights) may become very small, making it difficult to train the model effectively over long sequences.

Simple Example: Let's say we want to predict the next letter in a word sequence:

- Input: ["a", "p", "p", "l"]
- RNN takes the sequence and predicts the next letter "e", thus completing the word "apple".

python

Copy code

```
import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import SimpleRNN, Dense

# Create a simple RNN model

model = Sequential([

    SimpleRNN(50, input_shape=(5, 1)), # 5 time steps, 1 feature at
    each step

    Dense(1) # Output layer

])

model.compile(optimizer='adam', loss='mse')
```

3. Convolutional Neural Networks (CNNs) for Time Series

- What are CNNs?

- CNNs are typically used for image data, but they can also be used for time series data by applying 1D convolutions. CNNs capture local patterns in data, which can be useful in identifying short-term trends in time series.
- **How CNNs Work**
 - CNNs use filters (or kernels) that slide over the input data to detect patterns.
 - For time series data, we use 1D filters that move along the sequence and pick up features like short-term trends or patterns.
- **Advantages of CNNs for Time Series**
 - They can efficiently capture local dependencies and are less prone to the vanishing gradient problem.
 - Unlike RNNs, CNNs don't have memory across time steps, so they are not directly affected by long sequence lengths.

Simple Example: Predict the next value in a series of numbers (e.g., predicting the next number in a time series of stock prices).

python

Copy code

```
import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Conv1D, Dense, Flatten

# Create a simple CNN model for time series

model = Sequential([

    Conv1D(filters=64, kernel_size=2, activation='relu',
input_shape=(10, 1)), # 10 time steps, 1 feature

    Flatten(),

    Dense(1) # Output layer

])

model.compile(optimizer='adam', loss='mse')
```

4. When to Use RNNs vs CNNs

- **Use RNNs** when:
 - The sequence has long-term dependencies (e.g., predicting stock prices based on months of historical data).
 - There is a need to model memory or context over time.
- **Use CNNs** when:
 - The focus is on detecting short-term patterns or local trends (e.g., identifying patterns in a few time steps of a time series).
 - You need a faster model that doesn't rely on maintaining long-term dependencies.

5. Simple Comparison and Conclusion

- **Comparison:**
 - RNNs maintain memory across the sequence but are prone to the vanishing gradient problem.
 - CNNs detect short-term patterns efficiently but don't maintain memory.

Final Thoughts:

- Both RNNs and CNNs can be used for time series and sequence models, but the choice depends on the type of problem you are solving.
 - For problems requiring memory of past inputs, RNNs are better suited. For problems focusing on local patterns, CNNs may be more efficient.
-

Concept	RNN (Recurrent Neural Network)	CNN (Convolutional Neural Network)
Purpose	Designed for sequential data, such as time series and text.	Used for pattern recognition in data, often images, but can also be applied to sequences like time series.

Architecture	Has loops in the network to maintain information (hidden states) across time steps.	Uses convolutional layers to extract features through filters, emphasizing spatial or temporal patterns.
Focus	Sequence processing; keeps track of temporal dependencies between data points (e.g., predicting next word in a sentence).	Feature extraction; identifies local features and patterns in data, even across time (e.g., detecting patterns in time series).
Strength	Good at capturing long-term dependencies in sequences.	Efficient at recognizing localized patterns in data.
Limitation	Struggles with long sequences due to vanishing gradients.	Does not inherently maintain temporal order but can recognize patterns at different time scales.
Common Layers	LSTM (Long Short-Term Memory), GRU (Gated Recurrent Unit)	Convolutional layers, pooling layers
Example Task	Predicting the next value in a stock price sequence.	Detecting specific trends in sensor data or time series with high noise.

Identifying When to Use Trailing vs. Centered Windows in Time Series Modeling

When working with time series data, windows are often used to create input sequences for machine learning models, especially in forecasting or sequence prediction tasks. A "window" refers to a subset of the time series data that you use to make predictions about future values. There are two main types of windows: **trailing windows** and **centered windows**.

Let's explore when to use each type:

1. Trailing Windows

Definition: A trailing window includes data points leading up to the point where you want to make a prediction. It uses the most recent past values to predict the next point in the sequence.

When to Use Trailing Windows

- **When Predicting Future Values:** Trailing windows are most commonly used in scenarios where you want to predict a future value based on historical data. Since you are only using past information, it's a natural choice for forecasting tasks.
- **Causal Time Series Forecasting:** If you are dealing with a real-time prediction task (e.g., predicting the stock price for the next day based on the last 7 days), trailing windows are ideal because they only use past data.
- **Real-World Sequential Data:** Trailing windows are used when predictions are made in real-time and future data is unknown or unavailable. For example, in weather forecasting or sales forecasting, you use the latest data to predict future events.

Example of Trailing Window:

If you are predicting the temperature for Day 10 based on the past 7 days, your trailing window would look like this:

- Window: [Day 3, Day 4, Day 5, Day 6, Day 7, Day 8, Day 9]
- Predict: Day 10's temperature

python

Copy code

```
# Example for Trailing Window
# Consider time series data with 10 data points, predicting the next
value using past 5 data points
window = [data[5], data[6], data[7], data[8], data[9]]
```

2. Centered Windows

Definition: A centered window takes data points around a specific target, meaning you use both past and future values of a sequence to predict the current value. The window is centered around the point you're trying to predict.

When to Use Centered Windows

- **Smoothing and Denoising:** Centered windows are commonly used when you need to smooth or denoise data, such as when applying a moving average or in signal processing tasks.
- **Post-Event Analysis:** If you are analyzing data where the full series is available (e.g., post-hoc analysis, where the data is complete), centered windows help to smooth out fluctuations by looking at both past and future values.
- **Retrospective Models:** When the task allows for the use of future data (e.g., offline analysis of completed data like smoothing a time series after it's been fully observed), centered windows can be applied.

Example of Centered Window:

If you're using a centered window to predict the value at Day 6, you might take the following window:

- Window: [Day 3, Day 4, Day 5, Day 6, Day 7, Day 8, Day 9]
- Predict: Centered prediction (e.g., smoothing or averaging) around Day 6

python

Copy code

```
# Example for Centered Window
# Consider time series data with 9 data points, using centered windows
around the target
centered_window = [data[2], data[3], data[4], data[5], data[6],
data[7], data[8]]
```

Summary

- **Trailing Windows** are best for **forecasting** and real-time applications where you only have access to past data.
- **Centered Windows** are useful for **smoothing** or retrospective analysis, where future data is available and helps to make more accurate predictions or trends visible.

Practical Example:

Imagine you're forecasting sales data. If you want to predict next month's sales based on historical data, you would use a **trailing window** because you don't have future sales data. But if you want to smooth out existing sales trends using past and future data points, you would use a **centered window**.

Each window type is chosen based on the problem's constraints (whether future data is available) and the model's purpose (prediction or smoothing).

Let's go through an example of **smoothing** using the **centered window concept**. We'll work with some **noisy temperature data** and demonstrate how a centered window helps smooth out the fluctuations.

Example: Smoothing Daily Temperature Data

Imagine we have recorded the daily temperature for 7 days, but the data is noisy due to sensor issues. To get a clearer trend, we want to smooth the temperatures using a centered window of size 3.

Noisy Temperature Data (in Celsius):

- Day 1: 21°C
- Day 2: 22°C
- Day 3: 20°C
- Day 4: 30°C (anomaly due to sensor error)
- Day 5: 23°C
- Day 6: 21°C
- Day 7: 19°C

The goal is to reduce the impact of the anomaly on Day 4 by averaging the temperatures before and after it.

Step-by-Step Process for Smoothing Using a Centered Window:

1. Centered Window Concept:

For smoothing the temperature of **Day 4**, we'll use data from both **before and after Day 4**. If we use a centered window of size 3, we average the temperatures from **Day 3, Day 4, and Day 5**.

- **Window for Day 4:** [Temperature on Day 3, Temperature on Day 4, Temperature on Day 5]

2. Apply Centered Window Smoothing:

We smooth each day's temperature (except for the first and last days, where we don't have enough data for a full window) by averaging the values in the window.

Smoothing for Day 4:

- **Window:** [Day 3: 20°C, Day 4: 30°C, Day 5: 23°C]
- **Smoothed Temperature for Day 4:** $(20^{\circ}\text{C} + 30^{\circ}\text{C} + 23^{\circ}\text{C}) / 3 = 24.33^{\circ}\text{C}$

Here, the extreme temperature spike on Day 4 is **reduced** by the averaging process, leading to a smoothed value of 24.33°C.

Smoothing for Other Days:

- **Day 3:** Average of Day 2, Day 3, and Day 4 $\rightarrow (22^{\circ}\text{C} + 20^{\circ}\text{C} + 30^{\circ}\text{C}) / 3 = 24^{\circ}\text{C}$
- **Day 5:** Average of Day 4, Day 5, and Day 6 $\rightarrow (30^{\circ}\text{C} + 23^{\circ}\text{C} + 21^{\circ}\text{C}) / 3 = 24.67^{\circ}\text{C}$

3. Smoothed Data:

After applying the centered window, the smoothed temperatures look like this:

- Day 2: 21.67°C (smoothed using [Day 1, Day 2, Day 3])
- Day 3: 24°C
- Day 4: 24.33°C
- Day 5: 24.67°C
- Day 6: 21°C (smoothed using [Day 5, Day 6, Day 7])

Conclusion:

Using a centered window has helped us smooth out the noisy temperature spike on Day 4. Instead of the anomaly at 30°C, we get a smoothed value that is closer to the general trend of the other days.

This approach is especially useful in applications like weather forecasting, where you might want to smooth out unexpected spikes or dips caused by measurement errors.

Code Example:

python

Copy code

```
import numpy as np
```

```
# Original noisy temperature data
```

```
temperatures = np.array([21, 22, 20, 30, 23, 21, 19])
```

```
# Apply centered window smoothing with a window size of 3
```

```
smoothed_temperatures = np.convolve(temperatures, np.ones(3)/3,  
mode='same')
```

```
print(smoothed_temperatures)
```

Output:

csharp

Copy code

```
[14.33 21.00 24.00 24.33 24.67 21.00 13.00]
```

In this output, you can see how the smoothing process adjusts the temperature values to reduce noise and create a more consistent trend over the data.

1. Introduction to Sequence Bias (10 minutes)

- **Definition:** Sequence bias occurs when a model gives undue importance to certain sequences in the training data, leading to skewed predictions. This can happen due to patterns in the data, such as an increasing trend, seasonality, or repeated patterns that the model learns too well, resulting in overfitting.
- **Examples:**
 - **Example 1:** Imagine predicting stock prices. If the model is trained on data where stock prices always go up, it may become biased and predict an upward trend even when the market is falling.
 - **Example 2:** Predicting sales data where seasonal effects are prominent, leading the model to overly rely on these seasonal patterns.

2. Understanding Sequence Bias with a Simple Example (15 minutes)

- **Scenario:** Consider a simple dataset of monthly sales figures for a small shop over two years. The sales have a seasonal pattern, with spikes during festive seasons.

Example Data:

python

Copy code

```
sales_data = [50, 55, 60, 90, 95, 100, 70, 65, 60, 85, 90, 95,
              50, 55, 60, 90, 95, 100, 70, 65, 60, 85, 90, 95]
months = list(range(1, 25))
```

-
- **Explanation:**
 - **Bias Issue:** If the model is trained on this data without any adjustments, it might predict high sales for every festive season, regardless of other influencing factors.
- **Activity:**
 - Plot the sales data and ask students to identify where they think the model might become biased (e.g., around the festive seasons).

3. Techniques to Identify Sequence Bias (10 minutes)

- **Visualization:**
 - Use plots to visualize data trends, seasonality, and any anomalies. This helps in identifying parts of the data where bias might occur.
- **Cross-Validation:**
 - Explain how splitting the data into training and validation sets helps in spotting sequence bias. If the model performs well on the training data but poorly on validation data, sequence bias might be present.

4. Compensating for Sequence Bias (15 minutes)

- **Techniques:**
 - **Data Augmentation:** Modify the training data by adding noise or shifting sequences to make the model less reliant on specific patterns.
 - **Normalization:** Scale the data to remove trends and seasonality, making the model focus on relative changes rather than absolute values.

Min-Max Normalization is a technique used to scale data to a fixed range, typically [0, 1] or [-1, 1]. This technique is particularly useful in machine learning models that rely on distance calculations, such as neural networks, where large differences in scale between features can negatively impact model performance.

How Min-Max Normalization Works

Min-Max Normalization rescales the original data to fit within a specific range by using the following formula:

$$x' = \frac{x - \min(X)}{\max(X) - \min(X)}$$

Where:

- x is the original data point.
- x' is the normalized data point.
- $\min(X)$ is the minimum value in the dataset.
- $\max(X)$ is the maximum value in the dataset.

- **Regularization:** Introduce penalties for overly complex models to prevent overfitting to specific sequences.

Regularization is a technique used to prevent a model from becoming too complex and overfitting the training data. Overfitting occurs when a model learns not just the underlying patterns in the data but also the noise or irrelevant details. This leads to poor generalization to new, unseen data.

How Regularization Works

Regularization introduces a penalty to the model's loss function (the function it tries to minimize during training). This penalty discourages the model from fitting too closely to the training data by penalizing large weights or overly complex models.

Types of Regularization

1. L1 Regularization (Lasso):

- Adds the absolute value of the coefficients as a penalty to the loss function.
- Encourages sparsity in the model (i.e., some weights become zero), which can lead to simpler models.

2. L2 Regularization (Ridge):

- Adds the square of the coefficients as a penalty to the loss function.

- Encourages smaller weights, preventing the model from relying too heavily on any single feature.

3. **Dropout Regularization:**

- Randomly "drops out" (sets to zero) a subset of the nodes in a neural network during training.
- Prevents the network from becoming too dependent on specific nodes, encouraging it to learn more robust features.